



Xi'an Jiaotong-Liverpool University

西交利物浦大学

MFE113TC

Programming Foundation for Intelligent Manufacturing Engineering

Lecture 08 Structures in C and Classes in C++

Dr. Jiachen Liu

jiachen.liu@xjtlu.edu.cn

Fall, 2026

Outline

- Structures (**struct**) in C/C++;
 - Structures as Arguments;
 - Linked List;
- Class and Object;
 - CRC Methods;
 - Class Implementation;
 - Separated Compilation;
 - Unified Modelling Language

Structures (**struct**) in C/C++

What and Why

- Most objects in the world are identified by a collection of data
 - where each member data defines one aspect of the object;
- Each member data may be a different type
 - E.g., you as a person may be defined by:
 - Name (**character string**);
 - ID number (**integer**) —> (**character string**);
 - Age (**integer**);
 - Job (**character string**);
 - etc...

Structures (**struct**) in C/C++

Declaring a structure

- Structures are declared with the keyword **struct**;
- The keyword can be followed by an optional name (e.g., book), to identify the form of the structure;
- The content of the structure is then defined in {};
 - a list of member declarations;
 - each declaration is terminated by a semicolon (;)
- *It is similar with declaring a new datatype book.

```
#include<stdio.h>
struct book{
    char title[20];
    char author[20];
    float value;
};
/*Don't miss the ;*/
```

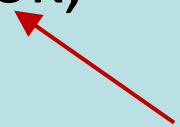
Structures (**struct**) in C/C++

Declaring a structure

- Now, we have a structure, we need to declare a variable of this type.

```
#include<stdio.h>
struct book{
    char title[20];
    char author[20];
    float value;
};
main(){
    struct book my_book;
    /*my_book is now a
    structure*/
```

```
#include<stdio.h>
struct book{
    char title[20];
    char author[20];
    float value;
}my_book;
main(){
    ...
    /*alternative declaration*/
```



Variable

Structures (**struct**) in C/C++

Declaring a structure with **typedef**

- For convenience, the keyword **typedef** can be used.
- Note: book comes a variable type and can be used like **int** instead of writing **struct** book.

```
#include<stdio.h>
typedef struct {
    char title[20];
    char author[20];
    float value;
} book;
```

```
main(){
    book name;
```

Structures (**struct**) in C/C++

Initializing a structure

- Initialization can be by a ',' separated list of value

```
#include<stdio.h>
struct book{
    char title[20];
    char author[20];
    float value;
};
main(){
    struct book my_book={
        "Dracula",
        "Bram Stoker",
        9.99};
```

Note: Do not assume the memory size of the structure is the sum of the sizes of its members. It is usually larger.

Use "sizeof(book)" to find actual size.

Structures (**struct**) in C/C++

Accessing a structure

- Use the dot (.) operator to refer to a structure member;
- Members can then be used like any other variables.

```
struct book{  
    char title[20];  
    char author[20];  
    float value;  
};  
main(){  
    struct book my_book={  
        "Dracula",
```

```
        "Bram Stoker",  
        9.99};  
    float price;  
  
    price = my_book.value;  
  
    printf("the book price is  
        %f",price);
```

Structures (**struct**) in C/C++

Filling / Accessing a structure

- The content of one book is input from the keyboard.

```
#include<stdio.h>
```

```
struct book{  
    char title[20];  
    char author[20];  
    float value;};
```

Declaration is not creation. book
ONLY describes structure content

Don't forget the ;

```
main(){
```

```
struct book lib;
```

Creation of object

```
gets(lib.title);
```

```
gets(lib.author);
```

```
scanf("%f",&lib.value);
```

```
printf("%s by %s is £%.2f\n", lib.title, lib.author, lib.value);}
```

Structures (**struct**) in C/C++

Filling / Accessing a structure

- The content of one book is input from the keyboard.

```
#include<stdio.h>
```

```
struct book{  
    char title[20],  
    char author[20];  
    float value;};
```

```
main(){  
    struct book lib;  
    gets(lib.title);  
    gets(lib.author);  
    scanf("%f",&lib.value);  
    printf("%s by %s is £%.2f\n", lib.title, lib.author, lib.value);}
```

Keyword **struct** identifies that the following is a structure variable

book is the name of the structure

Content can be a mix of as many variables and types as wanted (even another structure).

Structures (**struct**) in C/C++

Filling / Accessing a structure

- The content of one book is input from the keyboard.

```
#include<stdio.h>
struct book{
    char title[20];
    char author[20];
    float value;};

main(){
    struct book lib;
    gets(lib.title);
    gets(lib.author);
    scanf("%f",&lib.value);
    printf("%s by %s is £%.2f\n", lib.title, lib.author, lib.value);}
```

Create a structure of type book called lib

To gain access, use the name of the **struct** variable followed by the dot then the name of the member i.e. name.member

Structures (**struct**) in C/C++

Filling / Accessing a structure

- The content of one book is input from the keyboard.

```
#include<stdio.h>
struct book{
    char title[20];
    char author[20];
    float value;};

main(){
    struct book lib;
    gets(lib.title);
    gets(lib.author);
    scanf("%f",&lib.value);
    printf("%s by %s is £%.2f\n", lib.title, lib.author, lib.value);}
```

gets() copies a string from the keyboard and places it at an address. The member title is an array, remember the name of an array is the address of the first element

gets() is unsafe nowadays and should be replaced by fgets() in modern C.

[H][e][l][l][o][\0]

lib.title[0] → H

&lib.title[0] → H's address

lib.title → 1st element's address

Structures (**struct**) in C/C++

Filling / Accessing a structure

- The content of one book is input from the keyboard.

```
#include<stdio.h>
struct book{
    char title[20];
    char author[20];
    float value;};
main(){
    struct book lib;
    gets(lib.title);
    gets(lib.author);
    scanf("%f",&lib.value);
    printf("%s by %s is £%.2f\n", lib.title, lib.author, lib.value);}
```

scanf() is used to get a real number from the keyboard.
'&' is required since value is a float variable

member values are accessed with the dot (.) operator

gets() → read char
scanf("%f") → read float
scanf("%d") → read int

Structures (**struct**) in C/C++

Using a pointer to structures

- Yes... they are back again!
- Pointers allow a locally defined struct to be modified in a function;

```
main(){  
    char *title;  
    struct book my_book;  
    struct book *pbook;  
    pbook = &my_book;  
    title = pbook ->title;}
```

declaration uses the usual *

type is **struct** book

pbook is pointer name

& gives address of the
struct variable my_book

Access to members using pointers
requires

'->' or **(*pbook).title** not '.'

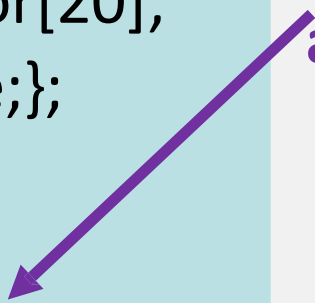
Structures (**struct**) in C/C++

Using a pointer to structures

```
#include<stdio.h>
struct book{
    char title[20];
    char author[20];
    float value;};

main(){
    struct book lib;
    struct book *plib=&lib;
    gets(plib->title);
    gets(plib->author);
    scanf("%f",&plib->value);
    printf("%s by %s is £%.2f\n", plib->title, plib->author,
        plib->value);}
```

plib is a pointer to the
address of lib



Note . and -> have the
highest precedence of
ALL operators

What about more than 1 book?

Structures (**struct**) in C/C++

Array of structures

```
#include<stdio.h>
```

```
struct book{  
    char title[20];  
    char author[20];  
    float value;};
```

```
main(){  
    struct book lib[10];  
    gets(lib[1].title);  
    gets(lib[1].author);  
    scanf("%f",&lib[1].value);  
    printf("%s by %s is £%.2f\n", lib[1].title, lib[1].author,  
                                                lib[1].value);}
```

Declaration statement
declares an array

Array starts at lib[0]. Array
elements accessed as usual i.e.
second structure lib[1]

Linked List

- Normally, program design requires many decisions before coding begins.
- We now introduce the linked list, a fundamental data structure.
- Suppose we want to create a program to store a list of favorite movies. Each movie contains different types of information, such as its title, director, year, and genre.
- A struct is suitable for representing one movie.
- But how many movie records do we need?
- We could use an array of structures.
- However, what problems do arrays present?
- The size of a traditional C array is fixed and normally has to be determined in advance.

Linked List

- Problem with an array of structures:
 - Arrays have a **fixed size** and cannot grow automatically.
 - Inserting or deleting elements may require shifting other elements.
- Do we have any other choices?
 - We can dynamically create new structures when needed using **malloc()**, allowing the data structure to grow dynamically.
- Any other difficulties?
 - To organize these dynamically allocated structures, we need to **link** them together.

Linked List

- Each time malloc() is called, **a pointer to the new memory element** will be created;
- These will not usually be created in any particular location order, therefore, a simple pointer cannot be used to move from one structure to the next;
- The linked list **includes a pointer variable inside the structure**, which is set to **point to the next structure in the list**.

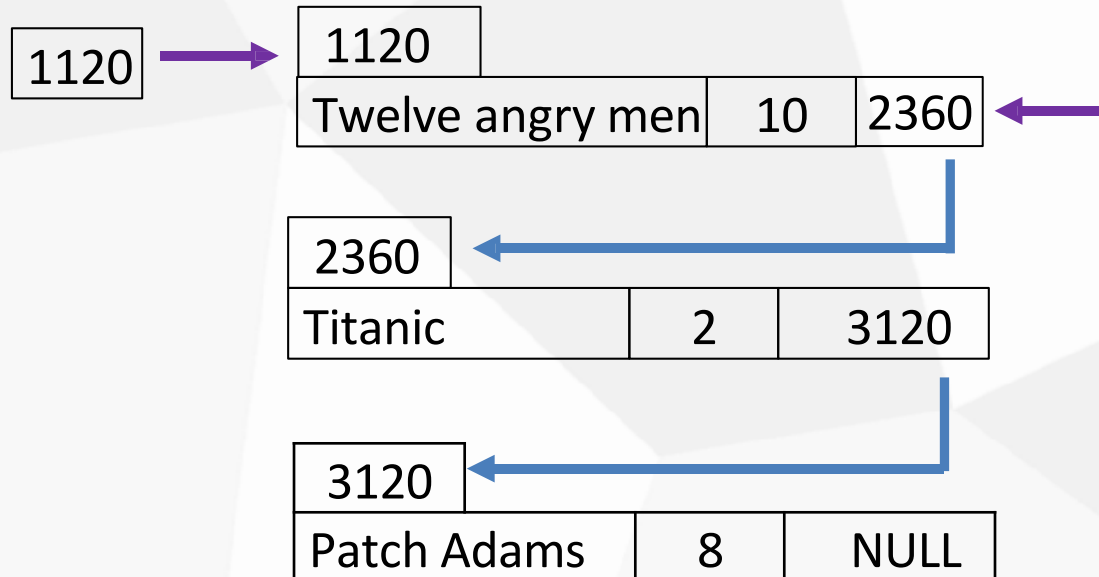


Lamb Shashlik

Linked List

```
struct movie{
    char * title;
    int rating;
    struct movie *next;
};
```

Pointer to
first structure



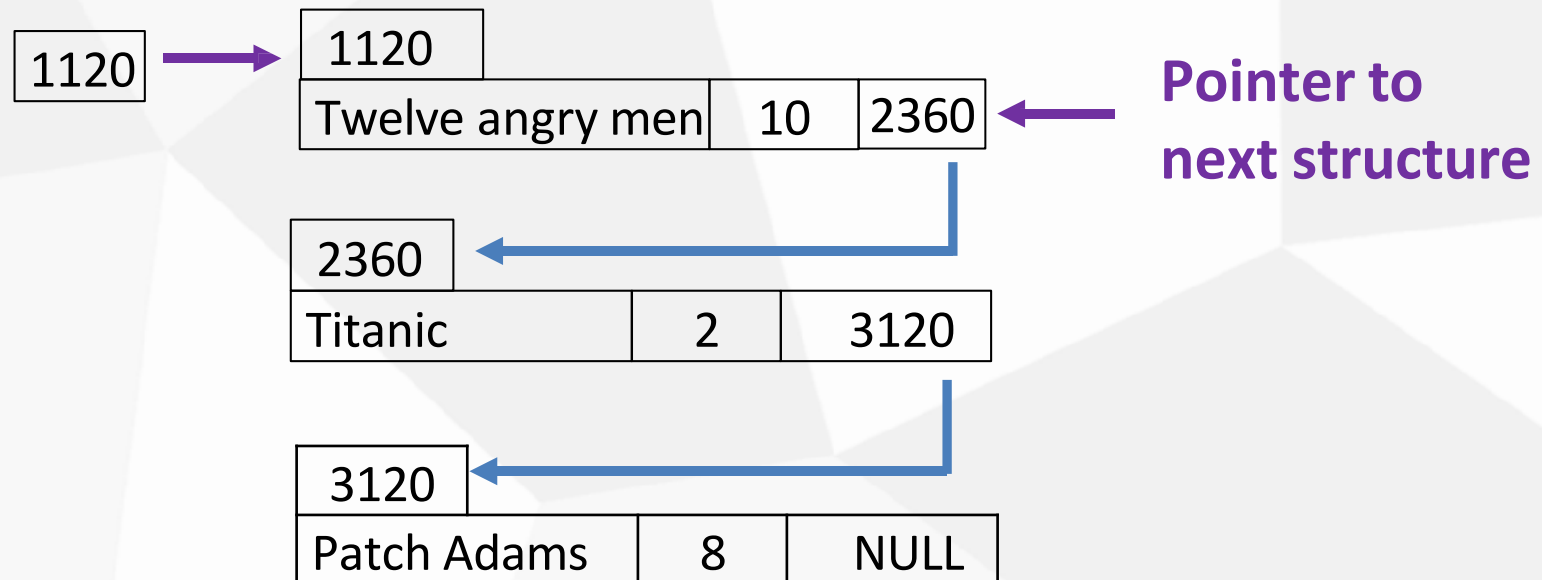
Pointer to
next structure



Linked List

- How could an element be inserted into or deleted from the list?

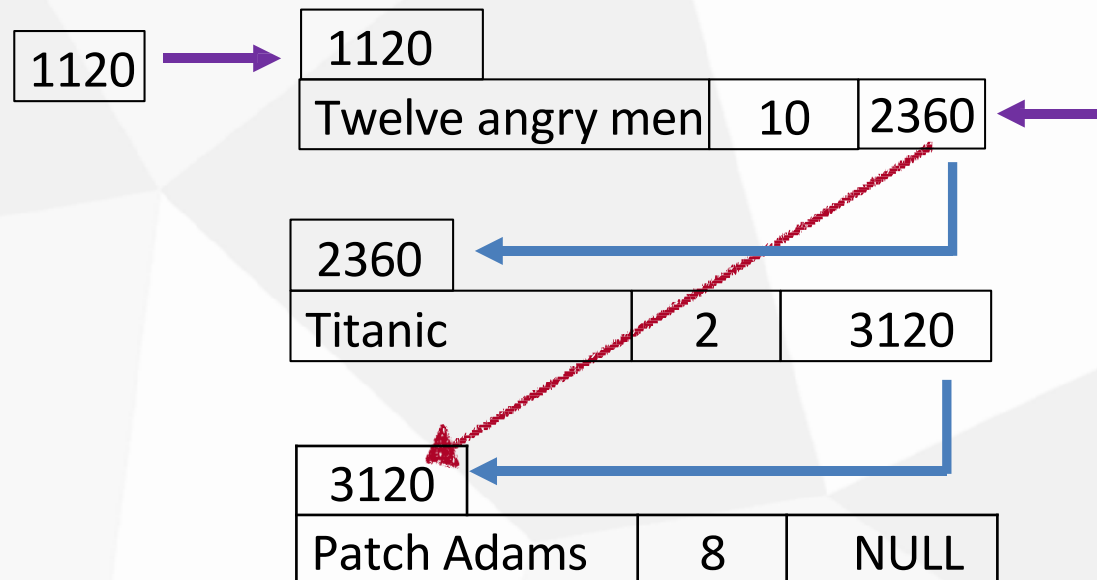
Pointer to
first structure



Linked List

- How could an element be inserted into or deleted from the list?

Pointer to
first structure



Pointer to next
structure

Linked List in C and C++

Feature	C	C++
Can use a linked list	Yes	Yes
Built-in linked-list container	No	Yes
Common implementation	struct + pointers + malloc()	struct / class + pointers, or STL
Dynamic memory management	malloc() / free()	new / delete, or standard containers
Standard library support	No standard linked-list container	std::list and std::forward_list
Typical use	Manual implementation	Manual implementation or standard library

Structure as Arguments in Functions

- Structures can be used as arguments:

```
myprint(mystructure);
```

- In this case, **a copy of the structure “mystructure”** is passed to the function “myprint”;
- This can be **slow** if the structure is embedded with many members or contains large arrays;
 - because **the copying consumes time and resources**;
- This is **different from passing an array to a function, which only passes a pointer.**

Structure as Arguments

```
#include<stdio.h>

struct funds{           /*declare the structure*/
    double current;
    double savings;
};

double sum(struct funds);

main(){                 /*function call sends copy*/
    struct funds ABC;    /*of ABC*/
    ABC.current = 1005;
    ABC.savings = 34.5;
    printf("Total funds in ABC %lf", sum(ABC));
}
```



Structure as Arguments

*/*function argument is a structure of same type*/*



```
double sum(struct funds my_funds){  
    double total;  
    total = my_funds.current + my_funds.savings;  
    return total;  
}
```

```
ABC.current = 1005;  
ABC.savings = 34.5;  
sum(ABC);
```

1039.5

Quiz

- True or False?
 - A **struct** is a data type of C;
 - A **struct** is used to declare a data type containing multiple fields (members);
 - Pointers can be used to access members of a structure;
 - An array cannot be a structure member.
- How is a linked list different from an array?

Quiz

- True or False?
 - A **struct** is a data type of C; **True**
 - A **struct** is used to declare a data type containing multiple fields (members); **True**
 - Pointers can be used to access members of a structure; **True**
 - An array cannot be a structure member. **False**
- How is a linked list different from an array?
 - **Array: stored in contiguous memory, usually fixed in size, and fast to access.**
 - **Linked list: nodes can be stored in different memory locations, connected by next pointers, and can grow or shrink dynamically.**

An array is connected by contiguous memory locations; a linked list is connected by pointers.

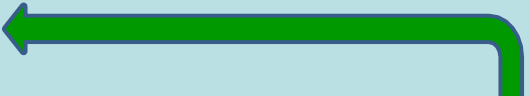


Break?

Class and Object

Recall of structure (**struct**)

```
#include<stdio.h>
struct book{
    char title[20];
    char author[20];
    float value;
};
```



/*Don't miss the ;*/

Feature	class	struct
Default member access	private	public
Can have variables	Yes	Yes
Can have functions	Yes	Yes
Can have constructors	Yes	Yes
Can have destructors	Yes	Yes
Can use inheritance	Yes	Yes
Can use public/private/protected	Yes	Yes

```
class Person
{
    int age;
};
```

```
Person p;
p.age = 20; // ✗
```

Feature	C	C++
struct	Yes	Yes
class	No	Yes
Member functions inside struct/class	No	Yes
public / private / protected	No	Yes
Constructors / destructors	No	Yes

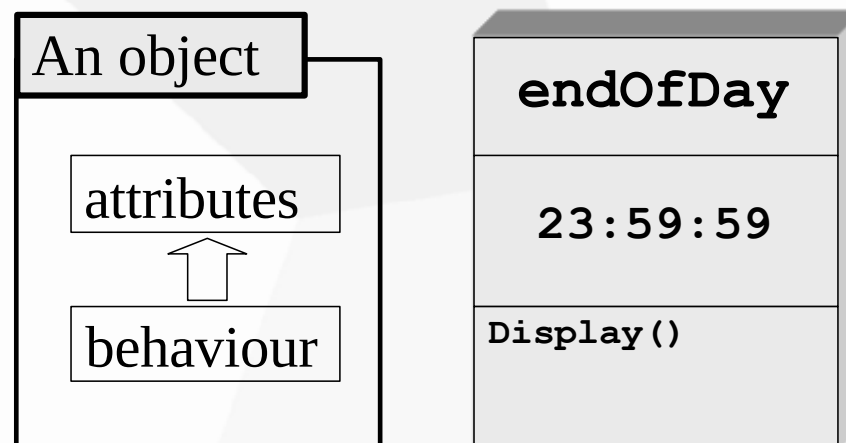
Class and Object

The idea of Objected-Oriented Design

- In the example, **each book can be regarded as an entity or object**:
 - Each object (book) is engaged with **several piece of information**;
 - Each object (book) can **perform some actions**, such as **telling or changing the corresponding information**.
- An object is embedded with:
 - **Data**: information about an object;
 - **Methods**: actions that an object can perform on its data.
- An object in **OOP (Object Oriented Programming)** is a component in the real or conceptual world that is mapped into the programming domain.

Class and Object Objects

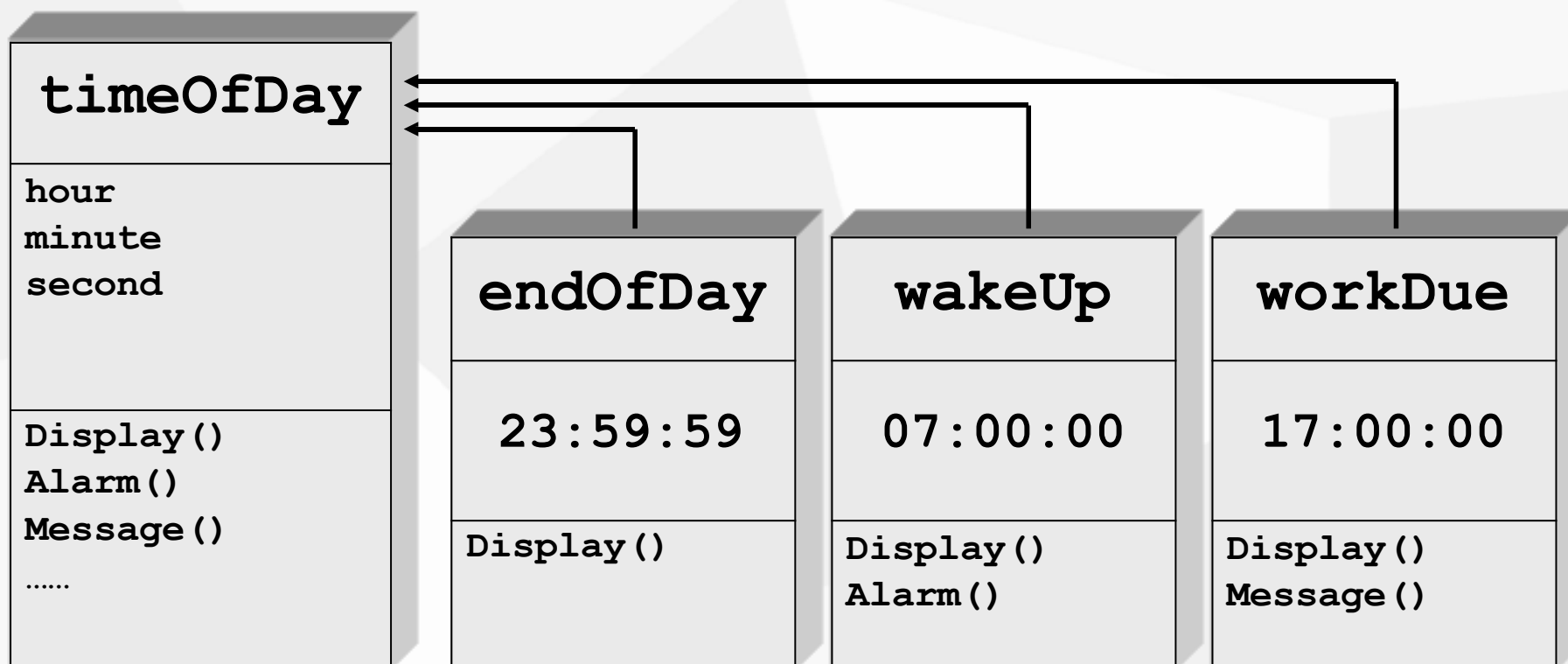
- An object in OOP (Object Oriented Programming) is a component in the real or conceptual world that is mapped into the programming domain.
- **Properties** or **attributes** or **data members**:
 - Necessary information required to describe the state of an object;
- **Methods** or **behaviour** or **function members**:
 - **All possible actions that an object should perform** in order to describe the corresponding behaviour in the problem specification.



Class and Object

Object -> Class

- A class is an **abstraction of all objects that share similar property structures and relevant methods:**



Class and Object

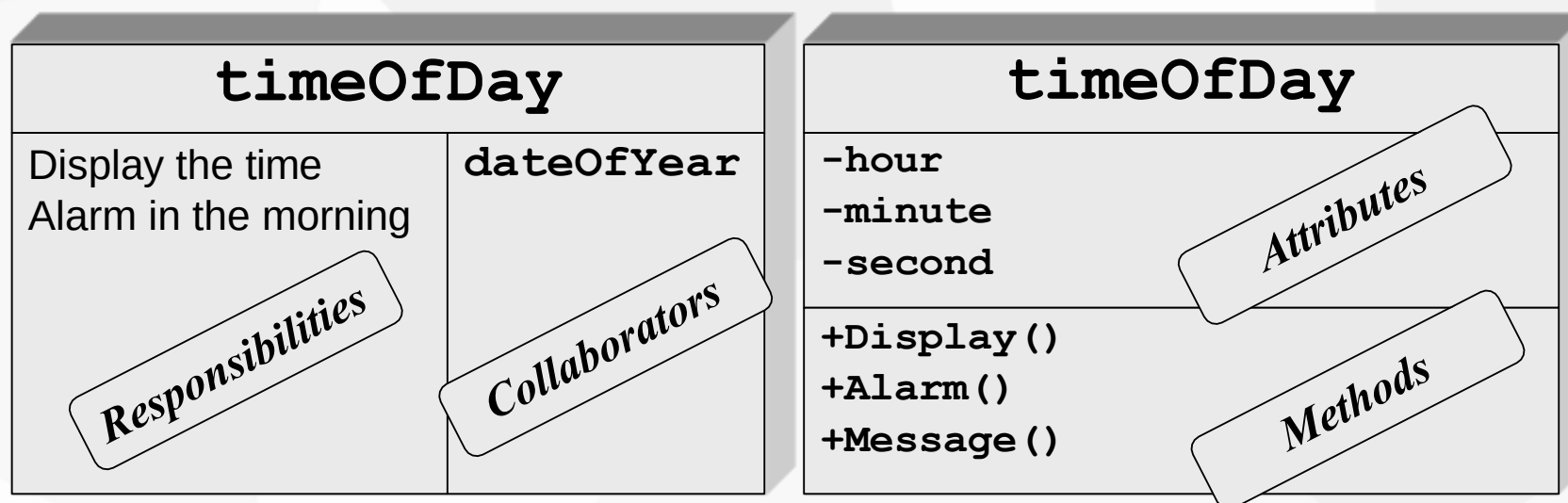
Class

- A class defines the structure of a group of **similar objects** (sharing the properties and methods);
- An object is an **instance** of class;
- Class is the **basic structure** in an **OOP** design;
- **Abstraction** can be easily achieved by using different classes;
- **Modularity** is automatically realised in programming based on classes since a class is **encapsulated** with **data (info) and actions (methods)**;
- **Information hiding** can be achieved by making data members private and providing controlled access through public member functions.
- Once defined, a class definition can be put into a **header file** and **repeatedly used** in any program (only if the header is included).

CRC Method

Introductions

- From the problem specifications, identify typical objects and possible classes (**C**lass);
- Identify the **responsibilities** (actions) for each class (**R**esponsibilities);
- For each of these classes, identify the **relevant classes** that are involved in the actions of that class (**C**ollaborators);
- Redefining the design by using use-cases.





CRC Method

Example

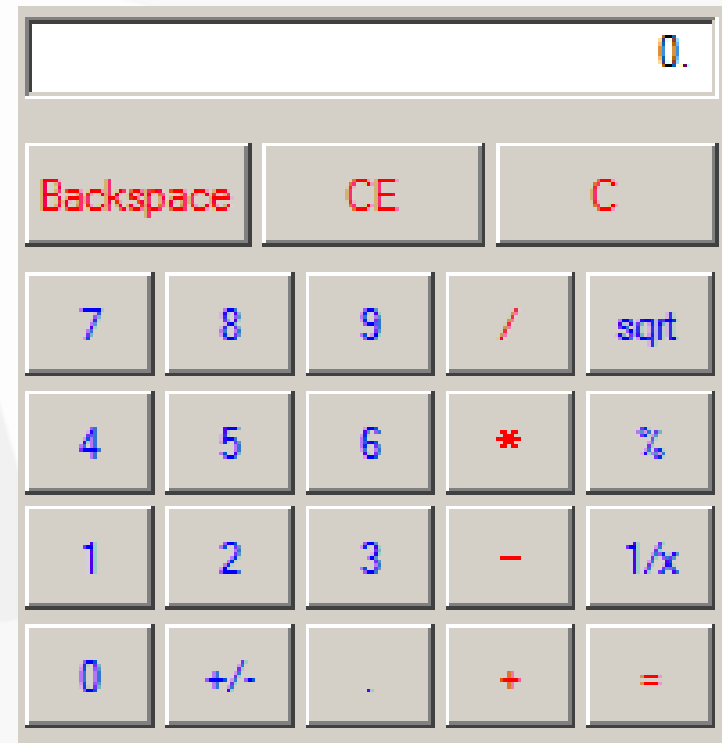
- Problem: to design a calculator that can perform addition, subtraction, multiplication, and division of two numbers. Such as:
 - $1+11=?$
 - $8\times 12=?$

CRC Method

Example to Use-Case

- Problem: to design a calculator that can perform addition, subtraction, multiplication, and division of two numbers. Such as:
 - $1+11=?$
 - $8\times 12=?$

Class: button
Class: display
Class: control unit



CRC Method

Example

Class: Button

Responsibilities:

Inform the control unit when a key is pressed and tell it what it presents.

Collaborators:

Control Unit



Class: Display

Responsibilities:

Display the graphic interface of calculator;
Display the input and the result.

Collaborators:

Control Unit



Class: Control

Responsibilities:

Get the inputs; Display the inputs and calculated results; Keep information and perform calculation.

Collaborators:

Button

Display



CRC Method

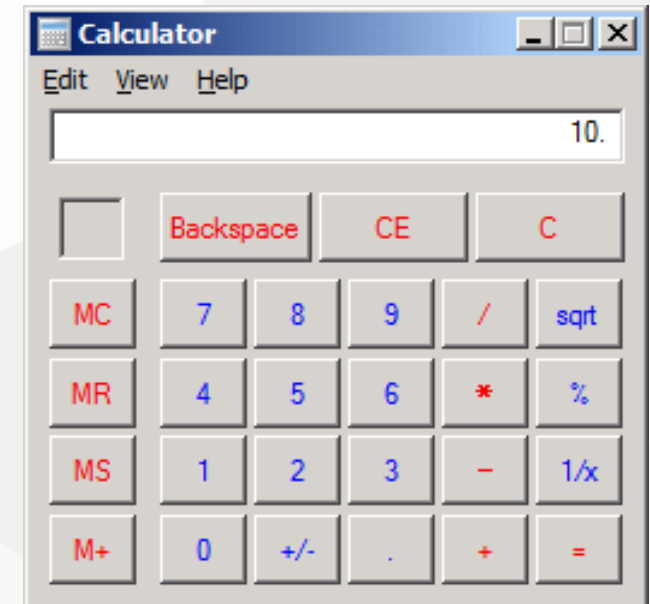
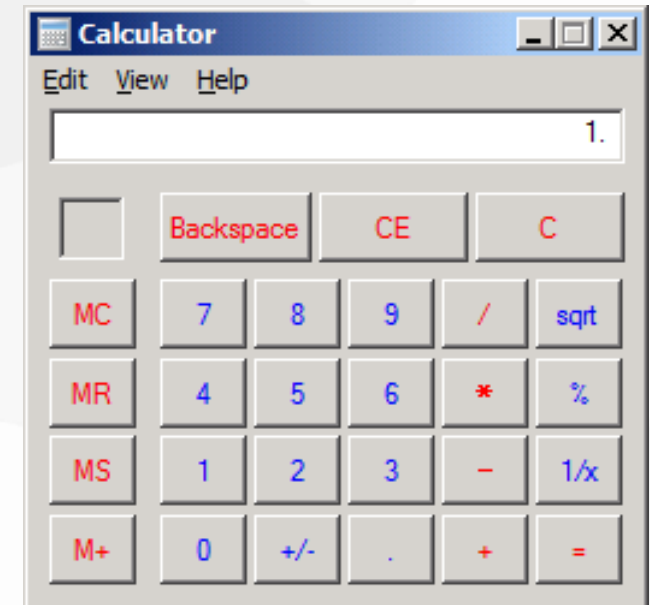
Walk through using Use-Cases

- Prepare a set of **typical cases**;
- Go through each case;
- Use-cases attempt to identify:
 - All objects involved in a typical case;
 - The information passed between objects;
 - The flow of controls.

CRC Method

Walk through using Use-Cases

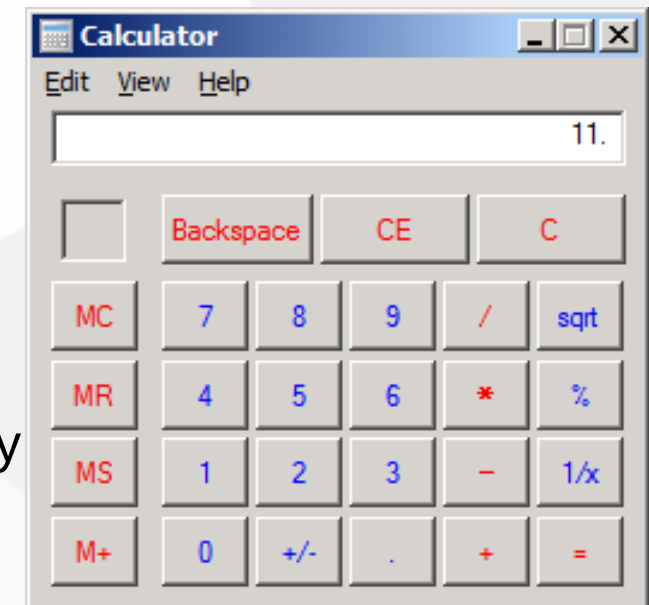
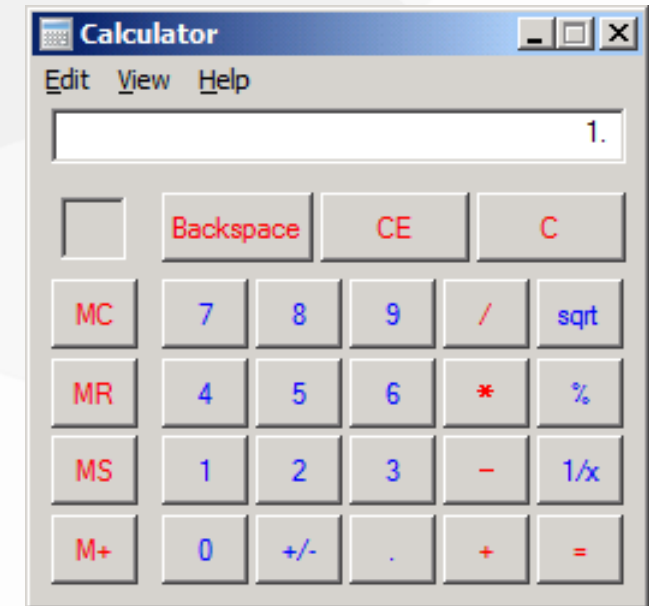
- Trace the Use-Case “10+1=?”
 - First, press 1, and the **1_button** tells the **control unit** that it has been pressed. The control unit **stores** 1 and tells the **display** to show 1;
 - Next, press 0, and the **0_button** tells the control unit that it has been pressed. The control unit stores 0, and **appends it to the previous digit**, and tells the display to **append 0 to display**;
 - Next, press +, and the **+_button** tells the control unit that it has been pressed. The control unit **stores the operator but do not display it**.



CRC Method

Walk through using Use-Cases

- Trace the Use-Case “10+1=?”
 - Next, press 1 again, and the 1_button tells the control unit that it has been pressed. The control unit **stores 1 in an area separated from the first number**, and tells the display to show 1;
 - **Implication**: the calculator clearly has to be able to store operand1, operator, and operand2;
 - Finally, press =, and the =_button tells the control unit that it has been pressed.
 - The control unit then performs the **calculation** “operand 1 operator operand2” and tells the display to show the answer.



CRC Method

Walk through using Use-Cases

- Identify some highlighted important aspects:
 - The button must be able to **get** the input from the user and **inform** the control unit of what the input is;
 - The control unit class must **keep track** of two operands and the operator, and do the **calculation accordingly**;
 - The display must be able to **display the input (as a string)** and **append a string to the current display**;
 - The interface protocol for various classes has been established.

Class Implementation in C++

Class definition

- Syntax:
 - Data member & Function member;
 - Private & Public.

private:
int age;

Person p;
p.age = 20; // ✗ p.setAge(20); // ✓

```
class ClassName
{
private:
    dataType _variableName;
    returnType _FunctionName(_parameterList);

public:
    dataType variableName;
    returnType FunctionName(_parameterList);

};
```

Private Cannot Be Used Directly

```
class Person
{
    private:
        int age;

    public:
        void setAge(int a);
        int getAge();
};
```

```
Person p;
p.age = 20; // ✗
```

```
p.setAge(20); // ✓
```

- A class contains data members and member functions. **private members are hidden from outside code**, while public members can be accessed from outside the class.

Class Implementation in C++

Member Function Declarations and Definitions

- The **declaration** of the member functions must be placed **inside** the class body;
- The **definition** of the member functions are **normally** placed **outside** the class body.

```
class Time
{
private:
    // data members
    int hour, minute, seconds;
public:
    // declaration of the methods
    void SetTime(int _h, int _m, int _s);
};

//definition of the method
void Time::SetTime(int _h, int _m, int _s)
{
    hour=_h;
    minute=_m;
    second=_s;
}
```

:: means scope resolution
operator

'SetTime()' belongs to 'Time'
class

We want to define the
'SetTime()' in 'Time'

Class Implementation in C++

Member Function Declarations and Definitions

- The **member functions** **can also** be defined inside the class body (we called as **inline functions**);
- During the compiling of the program, the compiler replaces the function call with the corresponding function code;
- It is faster than regular functions, but **coming with additional memory consumption.**

```
class Time
{
private:
    // data members
    int hour, minute, seconds;
public:
    // declaration and definitions of the methods
    void Time::SetTime(int _h, int _m, int _s)
    {
        hour=_h;
        minute =_m;
        second=_s;
    }
};
```

Class Implementation in C++

Declaration of the Class in the Header File

- Class **Time** is defined in the header file (**time.h**);
- We will tell the compiler what the class **Time** is by including the header file such as **#include "time.h"**;
- If we **omit** this, the compiler issues **error** messages wherever we use class **Time** and any of the related capabilities;
- In an **#include** directive, a header that you define in your program is placed in double quote (" "), rather than the angle brackets (< >) used for C++ Standard Library headers such as **<iostream>**;
- The **double quotes** in this example tell the compiler that the header is in the **same folder (recall relative path)**, rather than with the C++ Standard Library headers;
- You can **include headers into source-code files**, though you also may include them in other related headers.

Class Implementation in C++

```
class Person
{
private:
    int _id;
    string _name;
    int _office;
    int _nowhour;
public:
    void GetNowHour(void);
    void SetNowHour(string name, int workHour);
};

void Person::GetNowHour()
{
    cout << "Name: " << _name << endl;
    cout << "Worked " << _nowhour << " hours." << endl;
}

void Person::SetNowHour(string name, int workHour)
{
    _name = name;
    _nowhour = workHour;
}
```

Void here means no need to return anything, just execute the program.

```
void main()
{
    Person Haochuan;
    Haochuan.SetNowHour( "Haochuan", 996 );
    Haochuan.GetNowHour( );
}
```

But void main() is an outdated syntax now, use below instead:

int main()	int main()
{	{
...	...
return 0;	}
}	

Class Implementation in C++

```
void main( )      int main()  
{  
    Person Haochuan;           // Declaration of a class object  
    Haochuan.SetNowHour( "Haochuan",996 ); // Setup values  
    Haochuan.GetNowHour( );     // Check values  
}
```

- **Questions:** Can we initialize the data members when an object is declared? and How?
 - A class **constructor** is needed to initialize all data members when an object of a class is declared;
 - A class **constructor** is a member function.

Class Implementation in C++

Default Constructor

- The default constructor **uses the same class name** as the function name
- There is **no return type** in the default constructor;
- It will be **automatically called** when an object of the class is **declared**;
- No parameters / arguments need to be specified in the default constructor.



```
#include<iostream>
#include<string>
using namespace std;
class Person
{
private:
    int    _id;
    string _name;
    int    _office;
    int    _nowhour;

public:
    //default constructor declaration
    Person();
    int GetNowHour(void);
};
```

```
Person::Person()
{
    //default constructor definition
    _id      = 0;
    _name    = "N/A";
    _office  = 0;
    _nowhour = 0;
}
```

```
int Person::GetNowHour()
{
    return _nowhour;
}
```

```
int main()
{
    // declare the class object with the default constructor
    Person Haochuan;
    Haochuan.GetNowHour();
    printf("Hours:%d", Haochuan.GetNowHour());
    return 0;
}
```

public constructor means outside code can call it; as a member of the class, it can still access private data. So, we use Person::Person()

Class Implementation in C++

Normal/Parameterized Constructor

- Parameters / Arguments should be specified:
 - it differs from the default constructor by **collecting parameters** to get info.

Hours:996

```
#include<iostream>
#include<string>
using namespace std;
class Person
{
    private:
        int      _id;
        string    _name;
        int      _office;
        int      _nowhour;

    public:
        //normal constructor declaration
        Person(int id, string _firstName, int officeNumber, int workHour);
        int GetNowHour(void);
};

Person::Person(int id, string firstName, int officeNumber, int workHour)
{
    //normal constructor definition
    _id      = id;
    _name     = firstName;
    _office  = officeNumber;
    _nowhour = workHour;
}

int Person::GetNowHour()
{
    return _nowhour;
}

int main()
{
    // declare the class object with the normal constructor
    Person Haochuan(4, "Haochuan", 554, 996);
    Haochuan.GetNowHour();
    printf("Hours:%d", Haochuan.GetNowHour());
    return 0;
}
```

Class Implementation in C++

Example of Calling the Member Function

```
#include<iostream>
#include<string>
using namespace std;
class Person
{
    private:
        int    _id;
        string _name;
        int    _office;
        int    _nowhour;

    public:
        //default constructor declaration
        Person();

        //normal constructor declaration
        Person(int id, string firstName, int officeNumber, int workHour);
        void ShowNowHour(void);
};

Person::Person()
{
    //default constructor definition
    _id      = 0;
    _name    = "N/A";
    _office  = 0;
    _nowhour = 0;
}

Person::Person(int id, string firstName, int officeNumber, int workHour)
{
    //normal constructor definition
    _id      = id;
    _name    = firstName;
    _office  = officeNumber;
    _nowhour = workHour;
}

void Person::ShowNowHour()
{
    cout<<"Name: "<<_name<<endl;
    cout<<"Worked "<<_nowhour<< " hours.";
    cout<<endl;
}
```

Default Constructor

Normal Constructor

```
#include<iostream>
#include<string>

int main()
{
    Person Haochuan;
    Person Shan(1, "Shan Jaffry", 996);
    Haochuan.ShowNowHour();
    Shan.ShowNowHour();
    return 0;
}
```

Class Implementation in C++

Example of Calling the Member Function

```
// Example of a class member function
// Version 1: without any input argument
void Person::SetNowHour(void)
{
    int value;
    cout<<"Please input new value for nowhour for "<<name<<endl;
    cin>>value; _nowhour = value;
    cout<<"Actuion fiunished with "<<name<<"", and Thank you!"<<endl;
}
```

Same class can have many member functions. Even if the name is same as long as the parameters are different, it's allowed.

Since we don't have anything in SetNowHour(), we want user to input a value

```
// Example of ordinary member functions
// A member function changes the person's name
// Version 2: With a string type argument
void Person::SetNowHour(string _newName)
{
    name=_newName;
    cout<<"Name is changed to " <<name<<"", Thank you!"<<endl;
}
```

Class Implementation in C++

Building up the functionality gradually

Action(): like
a “menu”
here

```
// A member function to ask the user what to be done
// acting as the central control for all possible methods.

void Person::Action(void)
{
    int choice;
    string newName;
    cout<<"what do you want to do with "<<name<<endl;
    cout<<"1 for display, 2 for changing working hours, 3 for changing name:
";
    cin>>choice;
    if(choice==1)
        show_nowhour();
    else if(choice==2)
    {
        change_nowhour( );
        show_nowhour();
    }
    else if(choice==3)
    {
        cout<<"Please input the new name: ";
        cin>>newName;
        change_name(newName);
    }
}
```

Class Implementation in C++

Destructor (End its life when finished)

- To **free the memory allocated** for an object when it goes **out of the corresponding lifecycle** (scope, when the programme finishes using it):
 - Particularly **important** when using **dynamic memory allocation**;
 - It is a special function member of a class;
 - **~** + the name of the class;
 - No type needed in front of the function name;
 - No parameter / argument;
 - It is **called automatically** when an object goes out of the scope.

```
class Person
{
    private:
        int        _id;
        string      _name;
        int         _office;
        int         _nowhour;

    public:
        Person( );
        ~Person( );
};

Person::~~Person( )
{
    cout<<"Destructor called!"<<endl;
}
```

Class Implementation in C++

Class v.s. Structure

- An example of a complex number class

```
class ComplexClass
{
private:
    int _x, _y;
public:
    // Inline definition
    ComplexClass()
    {
        _x=0;
        _y=0;
    }

    void Set(int a, int b);
};

void ComplexClass::Set(int a, int b)
{
    _x=a;
    _y=b;
}
```

```
int main()
{
    int num=-9, num2=5;
    ComplexClass c1;
    c1.Set(num1, num2);
    c1._x+=1; // error
    c2._y--;
}
```

YES

No

- In the member function of the class, data members `_x` and `_y` can be **directly used** without the dot operator;
- Outside the class (e.g. in the main function), **the private data** members cannot be accessed.

Class Implementation in C++

Class v.s. Structure

- An example of a complex number structure

```
int main()
{
    int num=-9, num2=5;
    ComplexClass c1;
    c1.Set(num1, num2);
    c1._x+=1; // error
    c2._y--;  // error

    ComplexStruct s1;
    s1 = Set(num1, num2);
    s1._x+=1; // correct
    s1._y--;  // correct
    return 0;
}
```

```
struct ComplexStruct
{
    int _x, _y;
};

ComplexStruct Set(int a, int b)
{
    ComplexStruct _temp;
    _temp._x=a;
    _temp._y=b;
    return _temp;
}
```

- In the **external** function Set(), data members `_x` and `_y` **must be accessed using the dot operator**;
- Outside the structure, the data members can be accessed with the dot operator.

Class Implementation in C++

Class v.s. Structure

- These three functions are with the same names and parameter list, so will they cause conflicts?

```
class ComplexClass
{
private:
    int _x, _y;
public:
    // Inline definition
    ComplexClass()
    {
        _x=0;
        _y=0;
    }

    void Set(int a, int b);
};

void ComplexClass::Set(int a, int b)
{
    _x=a;
    _y=b;
}
```

```
struct ComplexStruct
{
    int _x, _y;
};

ComplexStruct Set(int a, int b)
{
    ComplexStruct _temp;
    _temp._x=a;
    _temp._y=b;
    return _temp;
}
```

```
double Set(int a, int b)
{
    double val;
    val = sqrt(a*a+b*b);
    return val;
}
```

- The 2nd and 3rd will in conflict, but the 1st won't.
- Because it is a member function of the ComplexClass, **which can only be called with the corresponding object of the respective class.**

Class Implementation in C++

Copy Constructor

- To use the data members of an existing object to **initialize an object of the same class when it is declared.**

```
class ComplexClass
{
    double _x, _y;

public:
    // Default constructor
    ComplexClass(){}

    // Normal constructor
    ComplexClass(double r, double i)
    {_x=r; _y=i;}

    // Copy constructor
    ComplexClass(ComplexClass &cNum)
    {_x=cNum._x; _y=cNum._y;}
}
```

```
int main()
{
    double num1=-9, num2=-5;
    ComplexClass c1;
    ComplexClass c2(num1, num2);
    ComplexClass c3(c2);
    return 0;
}
```

No reference sign when calling the copy constructor

Critical: using reference in parameter declaration

Class Implementation in C++

When the constructor / destructor will be called?

```
class ComplexClass
{
    double x;
    double y;
public:
    ComplexClass()
    {
        x=0;
        y=0;
        cout<<"Default constructor is called."<<endl;
    }

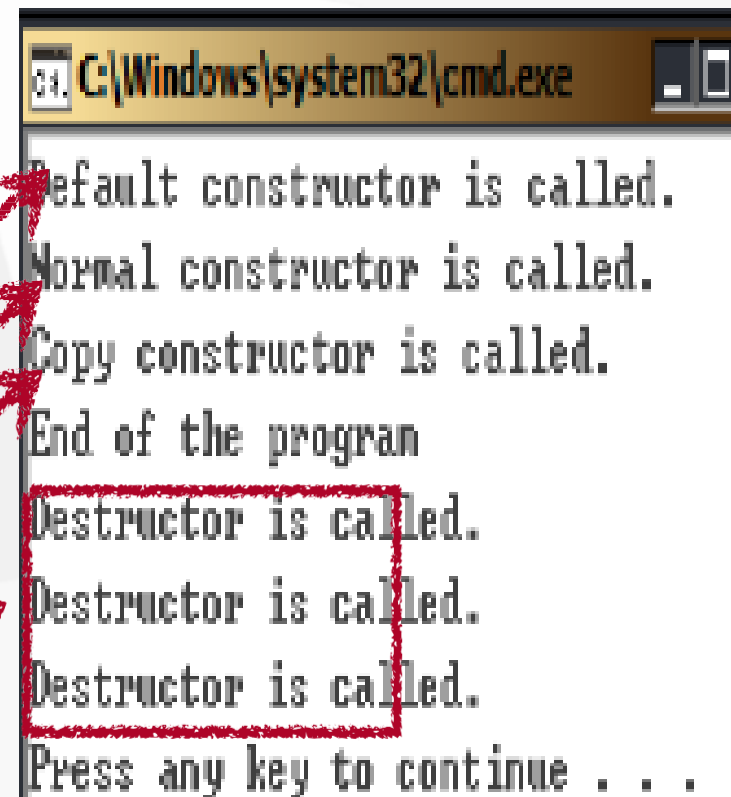
    ComplexClass(double r, double i=0)
    {
        x=r;
        y=i;
        cout<<"Normal constructor is called."<<endl;
    }

    ComplexClass(ComplexClass &cNum)
    {
        x=cNum.x;
        y=cNum.y;
        cout<<"Copy constructor is called."<<endl;
    }

    ~ComplexClass()
    {
        cout<<"Destructor is called."<<endl;
    }
};

void main()
{
    ComplexClass c1;
    ComplexClass c2(-9,5);
    ComplexClass c3(c2);
    cout<<"End of the program"<<endl;
}
```

- **Constructor** is called when 'ComplexClass' is called; **Destructor** is called when program is finished and out of the class scope.



```
C:\Windows\system32\cmd.exe
Default constructor is called.
Normal constructor is called.
Copy constructor is called.
End of the program
Destructor is called.
Destructor is called.
Destructor is called.
Press any key to continue . . .
```

Class Implementation in C++

When the constructor / destructor will be called?

```
class complexClass {...};

double complexClass::real(void) // get real part
{ return x;}

double complexClass:: imag(void) // get imaginary part
{ return y;}

bool isCEqual(complexClass cNum1, complexClass cNum2)
{
    bool temp;
    if((cNum1.real()==cNum2.real())&&(cNum1.imag()==cNum2.imag()))
        temp=1;
    else
        temp=0;

    cout<<"End of compare functions."<<endl;
    return temp;
}

void main()
{
    complexClass c2(-9,5);
    complexClass c3(9,5);
    if (isCEqual(c2,c3))
        cout<<"c2 equals to c3"<<endl;
    else cout<<"c2 doesn't equal to c3"<<endl;
    cout<<"End of the program"<<endl;
}
```

Copy c2 c3 to new
cNum1 and cNum2

C:\Windows\system32\cmd.exe

Normal constructor is called.

Normal constructor is called.

Copy constructor is called.

Copy constructor is called.

End of compare functions.

Destructor is called.

Destructor is called.

c2 doesn't equal to c3

End of the program

Destructor is called.

Destructor is called.

Press any key to continue . . .

Separated Compilation

*.h file and *.cpp file

- It is necessary to split the code **into separated files**:
- Reasons?
 - For separated files, **only the files changed will be recompiled**;
 - -> Less resource consumed during the compiling;
 - For group work, people will **code on individual files**.
- Files communicated through the inclusion of related header files.

Header files (*.h) contain	Source file (*.cpp) files contain
• Definition of classes; • Declaration of constants; • Declarations of nonmember functions; • Declarations of global variables	• Definitions of member functions; • Definitions of nonmember functions; • Definition of global variables.

Separated Compilation

*.h file and *.cpp file

```
/* Header file "point.h" */
#ifndef POINT_H
#define POINT_H
#endif

class Point
{
    int x,y;
public:
    Point();
    Point(int xx, int yy);
    Point (Point &p);
    void set(int xx, int yy)
    { x=xx;y=yy;}
    int GetX() const
    { return x;}
    int GetY() const
    { return y;}
};
```

```
/* Source file "point.cpp" */
#include<iostream>
#include"point.h"
using namespace std;

Point::Point()
{ x=0;y=0;}
Point::Point(int xx, int yy)
{ x=xx;y=yy;}
Point::Point(Point &p)
{ x=p.x;y=p.y;}

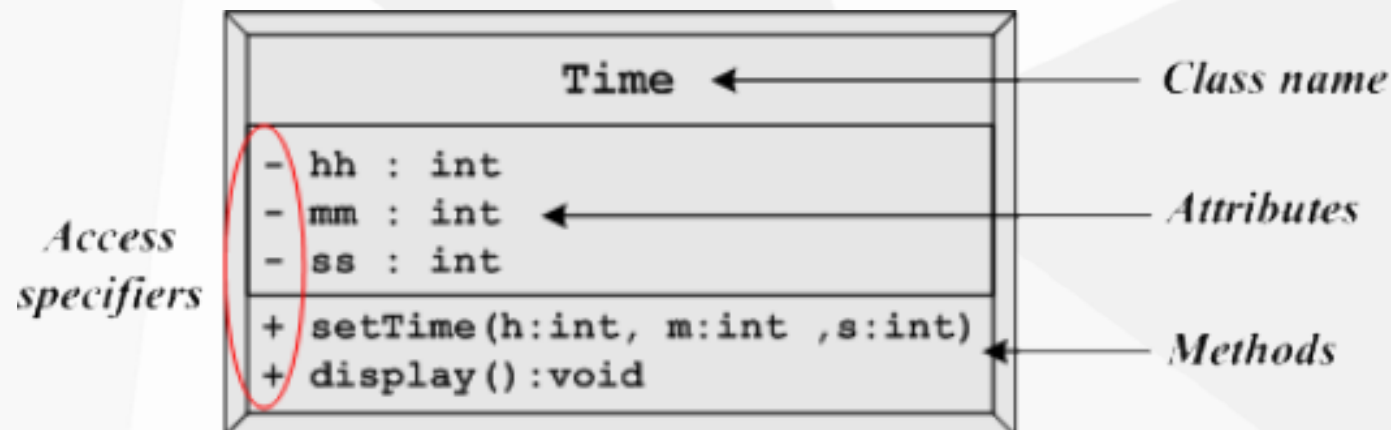
#include<iostream>
using namespace std;
#include"point.h"

int main()
{ Point myp1(1,1), myp2;
  return 0;
}
```

Unified Modelling Language

Introduction

- Unified Modelling Language (UML) was developed as a standardized general-purpose **graphical modelling language** in the field of object-oriented engineering:
 - UML includes a set of graphical notation techniques to create **visual models of object-oriented software-intensive systems**;
 - Class diagram: the rectangle illustrating a class is divided into **three compartments**, with the class name at the top, attributes (data members) in the middle, and methods (function members) at the bottom.



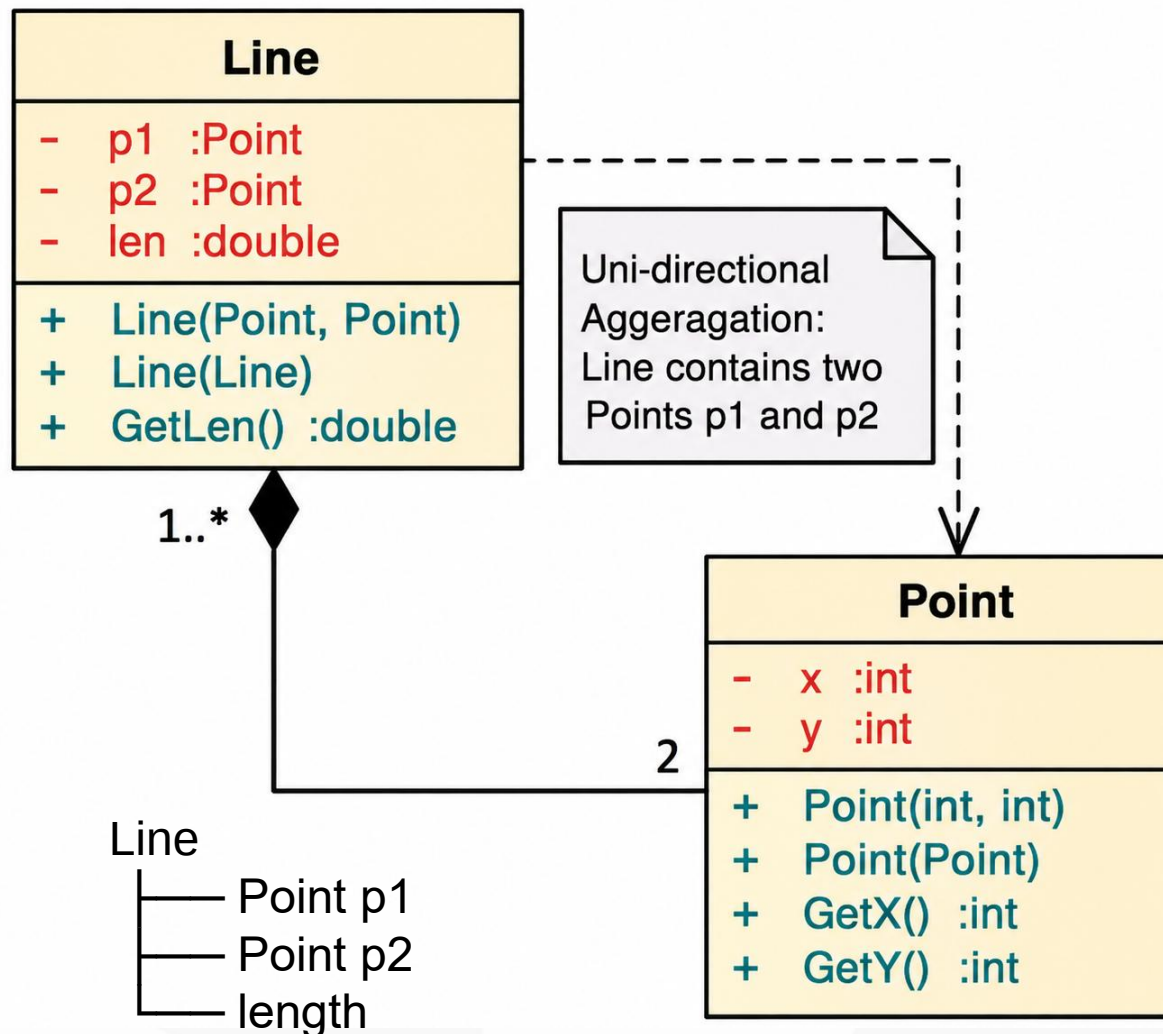
Unified Modelling Language

Several simple rules to draw UML:

- To distinguish the attributes from the **methods**, add parentheses after function names, such as **display()**;
- To indicate the data type of an attribute, use the UML format **attributeName:dataType**, for example, **hh:int**;
- To specify the parameter and return types of a method, place the type after each parameter name and the return type after the closing parenthesis. For example: `add(a:int, b:int) : int`. The return type may be omitted in simplified UML diagrams. **void** can be left out;
- Sometimes it is useful to illustrate the accessibility of the members. UML using **'-' for private members**, and **'+' for public members**.

Unified Modelling Language

UML to illustrate the class composition



class Point

```
{
private:
    int x;
    int y;

public:
    Point(int, int);
    Point(const Point&);
    int GetX();
    int GetY();
};
```

class Line

```
{
private:
    Point p1;
    Point p2;
    double len;

public:
    Line(Point, Point);
    Line(const Line&);
    double GetLen();
};
```

- A Line object contains two Point objects, p1 and p2. UML uses the relationship line and multiplicity 2 to show this class composition.

Special Use of Functions in C++

Function / Operator **Overloading**

- **Overloading**: using the **same function name** to create functions that perform different tasks:
 - **same function names**;
 - different **parameter / argument list**;
- Overloading functions that **perform closely related tasks** to improve the readability and understandability:
 - Do not overuse.

Valid	Invalid	Invalid	Invalid
<pre>int sum(float a, int b); int sum(int a, int b); int sum(int a, int b, int c); int sum(int a, int *b);</pre>	<pre>int sum(int a, int b); float sum(int a, int b);</pre>	<pre>int sum(int a, int b); int sum(int a, int &b);</pre>	<pre>int sum(int a, int b); int sum(int a, int b, int c=0)</pre>

Special Use of Functions in C++

Function / Operator Overloading

- Example 1: Normal Function Overloading

```
#include<iostream> using namespace std;

struct complex
{
    double real;
    double imag;
};

complex sum(complex a, double b)
{ a.real+=b; return a;}

complex sum(complex a, complex b)
{ a.real+=b.real;a.imag+=b.imag; return a;}

int main()
{
    double dNum=5.0;
    complex result, cNum={1,2}, cNum2={3,4};
    result=sum(cNum,dNum);
    result=sum(cNum,cNum2);
    cout<<result.real <<"+"i" <<result.imag<<endl;
    return 0;
}
```

Can overload if
the datatype is
different!

Special Use of Functions in C++

Function / Operator Overloading

- Example 2: Overloading of the Member Function

```
class complex
{
    double real;
    double imag;
public:
    complex(double r=0, double i=0)
    {    real=r; imag=i;}

    complex sum(complex a)
    {    complex temp;
        temp.real=a.real+real;
        temp.imag=a.imag+imag;
        return temp;}

    complex sum(double b)
    {    complex temp;
        temp.real=real+b;
        temp.imag=imag;
        return temp;}

    void display();
};
```

```
int main()
{
    double dNum=5.0;
    complex result1,result2;
    complex cNum(1,2),cNum2(3,4);
    result1=cNum.sum(dNum);
    result2=cNum.sum(cNum2);
    result1.display();
    result2.display();
    return 0;
}
```

complex = complex + complex

result2 = 4 + 6i

complex = complex + real

result1 = 6 + 2i

Function overloading allows multiple functions to have the same name but different parameter lists.

Special Use of Functions in C++

Operator Overloading

- How to calculate the addition, and subtraction of two complex numbers?

```
// Add function members to perform the arithmetic calculations
```

```
class complexClass
{
    double x;
    double y;
public:
    ..... 
};

complexClass(double a=0, double b=0)
{
    x = a; y = b;
}

void complexClass::assign(complexClass &a)
{
    x=a.x;
    y=a.y;
}

complexClass complexClass::plus(complexClass a)
{
    complexClass temp(a.x+x,a.y+y);
    return temp;
}

complexClass complexClass::minus(complexClass a)
{
    complexClass temp(x-a.x,y-a.y);
    return temp;
}

void complexClass::display()
{
    cout <<x <<(y>=0?"+":"")<<y <<"i"<<endl;
}
```

```
// Add function members
// to perform the arithmetic calculations
int main()
{
```

```
    complexClass a(3,2);
    complexClass b(5,4);
    complexClass result;
    result.assign(a.plus(b));
    result.display();
    result.assign(a.minus(b));
    result.display();
    return 0;
}
```

$a = 3 + 2i$
 $b = 5 + 4i$  $8+6i$
 $-2-2i$

How can I follow the **natural way** to do the calculation? i.e., $result=a+b$?
 $result=a-b$?

How can I follow the **natural way** to do the calculation? i.e., $\text{result} = a + b$?
 $\text{result} = a - b$?

- We can overload the + and - operators so that complex-number objects can be added and subtracted in the natural way.

```
complexClass complexClass::operator+(complexClass a)
{
    complexClass temp(x + a.x, y + a.y);
    return temp;
}
```

```
complexClass complexClass::operator-(complexClass a)
{
    complexClass temp(x - a.x, y - a.y);
    return temp;
}
```

- Then, you can use below in main body:

```
result = a + b;
result = a - b;
```

- This tells C++ that:
 $\text{result} = a.\text{operator}+(b);$
- b will be transmitted into the function.

Special Use of Functions in C++

Operator Overloading

- C++ tries to **make the user-defined data types behave in much the same way as that for built-in types**;
 - E.g., C++ permits the addition of two objects with the same syntax for basic types, such as **object1+object2**;
- The mechanism of providing such special approach to an operator is known as the **operator overloading**:
 - We can overload (offering additional meaning to) almost all the C++ operators;
 - We **cannot change the syntax** (operand, precedence, etc.)

Special Use of Functions in C++

Operator Overloading

- Syntax:

returntype keyword operator to be overloaded (parameter)

complexClass operator ± (complexClass a);

```
// Add function members to perform the arithmetic calculations
void complexClass::operator = (complexClass a)
{
    x=a.x;
    y=a.y;
}
complexClass complexClass::operator + (complexClass a)
{
    complexClass temp(a.x+x,a.y+y);
    return temp;
}
complexClass complexClass::operator - (complexClass a)
{
    complexClass temp(x-a.x,y-a.y);
    return temp;
}
```

```
// Use function members
int main()
{
    complexClass a(3,2);
    complexClass b(5,4);
    complexClass result;

    result=a+b;
    result.display();

    result=a-b;
    result.display();

    result=a-b-(a+b);
    result.display();

    return 0;
}
```

Special Use of Functions in C++

Default Arguments

- A default argument is **a value that is used automatically if you omit the corresponding actual argument from a function call**;
- The default value **should be set in the function declaration**.

- Syntax:

```
// In function declaration
returnType func_name(dataType param1, dataType param2 = value);

// In function definition
returnType func_name(dataType param1, dataType param2)
{
    statements;
}

// In calling function
func_name(arg1, arg2);
func_name(arg1);
// arg2 will be using the default value
// given in the declaration
```

Special Use of Functions in C++

Default Arguments

- Example 1: normal function with default argument

```
#include<iostream>
using namespace std;

double volume(double radius=1, double height=1);

int main()
{
    double a=5, b=2;

    cout<< volume() <<endl;    // omit 2 arguments
    cout<< volume(a) <<endl;   // omit 1 argument
    cout<< volume(a,b) <<endl; // doesn't omit any argument

    return 0;
}

double volume(double radius, double height)
{
    return 3.14*radius*radius*height;
}
```

Special Use of Functions in C++

Default Arguments

- Example 2: member function of a class with default argument

```
class complexClass
{
    double x;
    double y;
public:
    complexClass( ){}
    complexClass(double r, double i=0);
};

complexClass::complexClass(double r, double i)
{
    x=r;
    y=i;
}
```

```
int main( )
{
    complexClass c0;
    complexClass c1(-9);
    complexClass c2(-9,5);
    return 0;
}
```

Special Use of Functions in C++

Default Arguments

- Rules for using default arguments
 - When you use a function with an argument list, you must add default values **from the right to the left**.

```
int exch1(int n=1, int m=2, int k=3);    // VALID
int exch2(int n, int m=2, int k=3);    // VALID
int exch3(int n, int m, int k=3);      // VALID
int exch4(int n, int m=2, int k);      // INVALID
int exch5(int n=1, int m=2, int k);    // INVALID
int exch6(int n=1, int m, int k);     // INVALID
```

Special Use of Functions in C++

Default Arguments

- Rules for using default arguments
 - The actual arguments are assigned to the corresponding formal arguments from left to right. You cannot skip over arguments.

```
int exch1(int n=1, int m=2, int k=3);    // VALID
int exch2(int n, int m = 2, int k = 3);
int exch3(int n, int m, int k = 3);
temp = exch1(10,20,30); // VALID
temp = exch1(10, ,30);  // INVALID, cannot skip m
temp = exch1(10,20);    // VALID, equivalent to exch1(10,20,3)
temp = exch2(10);       // VALID, equivalent to exch2(10,2,3)
temp = exch3(10);       // INVALID, no value for m
```



Thanks